CSE 6109 — Complete Handnote

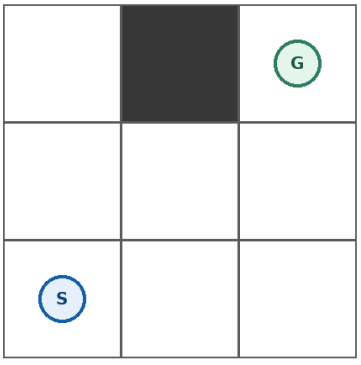
Automated Planning & Heuristic Search · Q1–Q7 solved step by step · exam-ready answers in yellow

Q1 BFS/DFS | Q2 A*/GBFS | Q3 PDDL | Q4 Abstraction | Q5 h^m | Q6 Relaxation | Q7 Landmarks | Last-night sheet

Q1 · Uninformed vs Informed search on a 3×3 grid

QUESTION 1 · [15 MARKS]

1. What is the difference between uninformed and informed state-space search? Given the following grid-world path-finding problem, perform the following tasks.



3x3 grid-world
S = start, G = goal, dark cell = wall (impassable)

MEANING What is being asked

Rows are numbered from the top (row 1 = top) and columns from the left. A state is a cell. Moves cost 1. Cell (1,2) is a wall, so 9 − 1 = **8 states**. You must list every state and its neighbours, run BFS and DFS by hand, then prove which algorithm is complete and optimal.

RULE Key definitions

- **Uninformed** search (BFS, DFS, UCS) uses only the problem definition, with **no h(n)**.
- **Informed** search (GBFS, A*) also uses a heuristic h(n) that estimates the distance to the goal.
- **Complete**: if a solution exists, the algorithm always finds one.

- **Optimal**: the solution it returns has minimum cost.
- **BFS** keeps its frontier in a FIFO queue (shallowest first). **DFS** uses a LIFO stack (deepest first).
- **Finite branching factor b** means every node has at most b children. A **finite graph** has finitely many states.

> **Q1 (first part)** What is the difference between uninformed and informed state-space search?

ANSWER Uninformed vs informed search

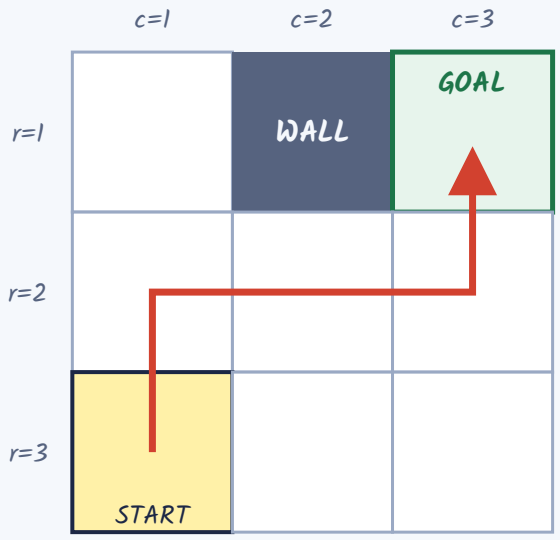
Informed search uses the Manhattan distance $h(r,c) = |r-1| + |c-3|$. It never overestimates, because a wall can only make the real path longer. So h is admissible, and it is also consistent.

Cell	(3,1)	(3,2)	(3,3)	(2,1)	(2,2)	(2,3)	(1,1)	(1,3)
h	4	3	2	3	2	1	2	0

	BFS	DFS	GBFS (f = h)	A* (f = g + h)
Uses h?	No	No	Yes	Yes
Frontier	FIFO queue	LIFO stack	priority queue by h	priority queue by g+h
Complete?	Yes (finite b)	Only finite graph + duplicate check	Yes on finite graphs (graph search)	Yes (positive costs)
Optimal?	Yes, unit costs only	No	No	Yes, if h is admissible (consistent for graph search without reopening)
On this grid	cost 4, 8 expansions	cost 4, 6 expansions	cost 4	cost 4, guaranteed

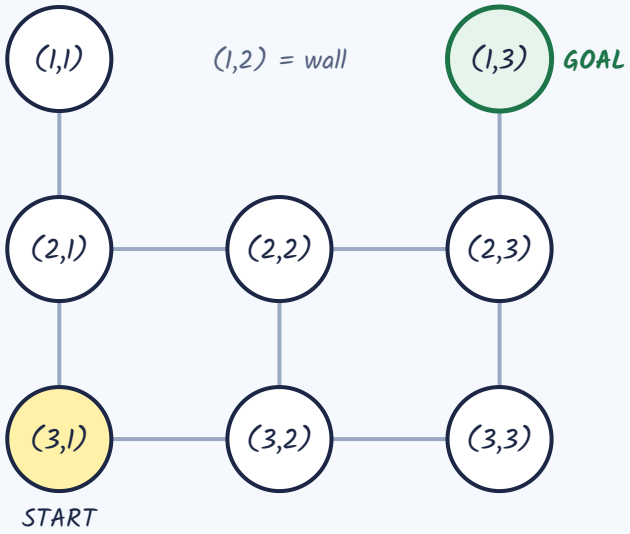
> **Q1 (i)** Represent the problem as a state-space graph over the 8 reachable cells (states), with an edge for every legal up/down/left/right move. Draw the complete graph and mark the start and goal states.

FIGURE (i) The grid and the state-space graph



Red = path found by BFS and DFS: (3,1) → (2,1) → (2,2) → (2,3) → (1,3), 4 moves

State	Position	Neighbours in tie-break order (Up, Right, Down, Left)
(3,1) START	bottom-left	Up → (2,1), Right → (3,2)
(3,2)	bottom-middle	Up → (2,2), Right → (3,3), Left → (3,1)
(3,3)	bottom-right	Up → (2,3), Left → (3,2)
(2,1)	middle-left	Up → (1,1), Right → (2,2), Down → (3,1)
(2,2)	centre	Right → (2,3), Down → (3,2), Left → (2,1) (Up = wall)
(2,3)	middle-right	Up → (1,3), Down → (3,3), Left → (2,2)
(1,1)	top-left	**Down → (2,1)** only (Right = wall) → dead end
(1,3) GOAL	top-right	Down → (2,3) only (Left = wall)



State-space graph: 8 states, 9 undirected edges (each edge = a legal move both ways, cost 1)

There are 9 undirected edges (18 directed moves): (1,1)–(2,1), (2,1)–(3,1), (2,1)–(2,2), (2,2)–(2,3), (2,2)–(3,2), (2,3)–(1,3), (2,3)–(3,3), (3,1)–(3,2), (3,2)–(3,3).

> **Q1 (ii)** Trace Breadth-First Search on this graph step by step, showing the frontier and the explored set after each expansion, until the goal is found. State the resulting path and its length.

STEPS (ii) BFS trace (FIFO queue, goal test when a node is expanded)

Step	Expand	Frontier after (front → back)	Explored
0	—	[(3,1)]	{ }
1	(3,1)	[(2,1), (3,2)]	{(3,1)}
2	(2,1)	[(3,2), (1,1), (2,2)]	{(3,1),(2,1)}
3	(3,2)	[(1,1), (2,2), (3,3)] ((2,2) already in frontier)	+ (3,2)
4	(1,1)	[(2,2), (3,3)] (dead end)	+ (1,1)
5	(2,2)	[(3,3), (2,3)]	+ (2,2)
6	(3,3)	[(2,3)]	+ (3,3)
7	(2,3)	[(1,3)]	+ (2,3)
8	(1,3) GOAL	goal test succeeds	all 8 states

Follow the parent pointers back: (1,3) ← (2,3) ← (2,2) ← (2,1) ← (3,1). **Path length 4, 8 expansions.**

> **Q1 (iii)** Trace Depth-First Search on the same graph (breaking ties by preferring Up, then Right, then Down, then Left). Does it find the same path? Explain, in terms of what each strategy guarantees, why BFS is complete and optimal for unit-cost graphs while DFS is only complete on finite graphs.

STEPS (iii) DFS trace (LIFO stack; push in reverse order L, D, R, U so that Up is popped first)

Step	Pop / expand	Stack after (top on the right)	Explored
0	—	[(3,1)]	{ }
1	(3,1)	[(3,2), (2,1)]	{(3,1)}
2	(2,1)	[(3,2), (2,2), (1,1)]	+ (2,1)
3	(1,1)	[(3,2), (2,2)] dead end, backtrack	+ (1,1)
4	(2,2)	[(3,2), (2,3)] ((3,2) already on stack)	+ (2,2)
5	(2,3)	[(3,2), (3,3), (1,3)]	+ (2,3)
6	(1,3) GOAL	goal test succeeds	6 states

DFS path: $(3,1) \to (2,1) \to (2,2) \to (2,3) \to (1,3)$, length 4, with only 6 expansions. It finds the **same path as BFS** here, but that is luck, not a guarantee.

PROOF (iii) Why BFS is complete and optimal, and DFS is not

BFS is complete (when b is finite). BFS expands every node at depth 0, then depth 1, then depth 2, and so on. If the shallowest goal is at finite depth d, only finitely many nodes ($\leq 1 + b + \ldots + b^d$) come before it, so BFS reaches it.

BFS is optimal for unit costs. When every edge costs 1, depth = $g(n)$. BFS finishes all of depth d before it touches depth $d+1$, so the first goal it expands is the shallowest one, which is also the cheapest. (With unequal edge costs BFS is **not** optimal. Use UCS or A^* instead.)

DFS is complete only on finite graphs, and only with a duplicate check (graph search). It can visit each state at most once, so it must eventually exhaust all reachable states. On an infinite graph, DFS can go down one infinite branch forever. **Tree-search** DFS (no explored set) can even loop forever on a finite graph that has cycles, for example $(2,1) \leftrightarrow (2,2)$.

DFS is not optimal. It commits to the first deep branch and returns the first goal it meets. It never compares that path with a shorter one it has not explored yet.

Memory trick: BFS = "Broad, Fair, Shortest" (unit costs). DFS = "Dives, Forgets Shortest". A = g (paid) + h (still to pay).*

EXAM-READY ANSWER

Difference: uninformed search (BFS, DFS, UCS) uses only the problem definition; informed search (GBFS, A*) also uses a heuristic h(n) that estimates the distance to the goal, so it usually expands far fewer nodes.

(i) 8 states (the wall (1,2) is removed); unit-cost moves U/R/D/L. Adjacency: (3,1): (2,1),(3,2) · (3,2): (2,2),(3,3),(3,1) · (3,3): (2,3),(3,2) · (2,1): (1,1),(2,2),(3,1) · (2,2): (2,3),(3,2),(2,1) · (2,3): (1,3),(3,3),(2,2) · (1,1): (2,1) · (1,3): (2,3).

(ii) BFS expands (3,1),(2,1),(3,2),(1,1),(2,2),(3,3),(2,3),(1,3), giving the path (3,1) → (2,1) → (2,2) → (2,3) → (1,3), cost 4 (length 4).

(iii) DFS expands (3,1),(2,1),(1,1)[dead end],(2,2),(2,3),(1,3), giving the same path, cost 4.

BFS is complete (finite b; it expands level by level) and optimal for unit costs (depth = cost, so the first goal found is the shallowest). DFS is complete only on finite graphs with duplicate detection and is not optimal (it returns the first goal it meets). Informed search (GBFS/A*) uses h (here Manhattan distance, which is admissible). A* with admissible h is optimal and usually expands fewer nodes than uninformed search.

Q2 · Consistent ⇒ Admissible; A* vs Greedy Best-First

QUESTION 2 · [15 MARKS]

2. *"A consistent heuristic is always admissible, but not vice versa" — justify this statement. Consider the weighted graph below for a delivery-robot path-finding problem, with edge costs and straight-line-distance heuristic values h(n) to the goal G given beside each node.*

```
Nodes: S, A, B, C, G
Edges (cost): S-A (2), S-B (5), A-C (4), B-C (1), A-G (7), C-G (2)
Heuristic h(n): h(S)=6, h(A)=4, h(B)=3, h(C)=2, h(G)=0
```

RULE Definitions

Admissible: $h(n) \le h^*(n)$ for every n (never overestimates)

Consistent: $h(u) \le c(u,v) + h(v)$ for every edge $u \to v$, and $h(goal) = 0$ (triangle inequality)

> **Q2 (first part)** *"A consistent heuristic is always admissible, but not vice versa" — justify this statement.*

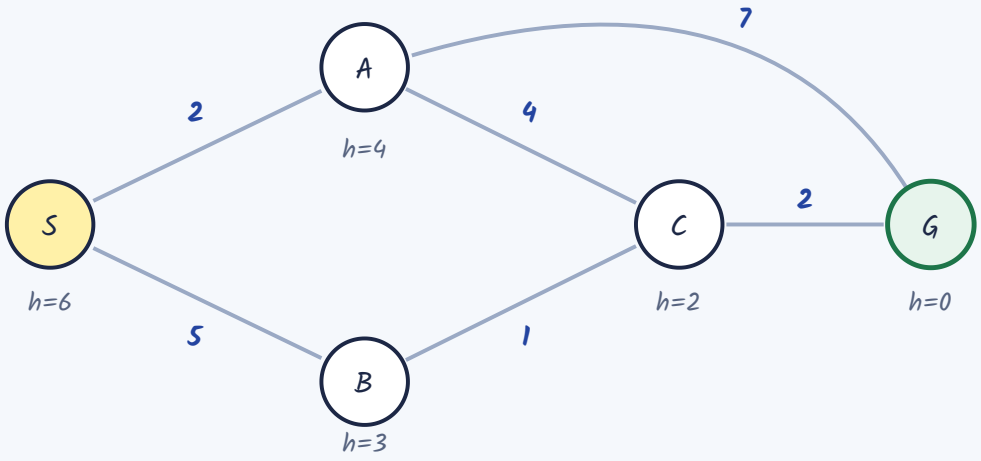
PROOF Consistent ⇒ admissible

Take any node n and an optimal path $n = v_0 \to v_1 \to \dots \to v_k = G$, so $h^*(n) = \Sigma\, c(v_i, v_{i+1})$. Apply consistency on each edge and chain the inequalities:

$$h(v_0) \le c(v_0,v_1) + h(v_1) \le c(v_0,v_1) + c(v_1,v_2) + h(v_2) \le \dots \le \Sigma_i\, c(v_i,v_{i+1}) + h(G) = h^*(n) + 0$$

So $h(n) \le h^*(n)$: h is admissible. If no goal is reachable from n, then $h^*(n) = \infty$ and the claim is trivially true.

Admissible but not consistent (counterexample). *Take $A \to B$ (cost 1), $B \to G$ (cost 10), with $h(A)=10$ and $h(B)=1$. Then $h^*(A)=11$ and $h^*(B)=10$, so $10 \le 11$ and $1 \le 10$: h is admissible. On edge $A \to B$, $h(A)=10 > c(A,B)+h(B) = 1+1 = 2$, so h is* ***not consistent****. Therefore admissible does not imply consistent.*



Blue numbers = edge costs (roads are two-way). Grey = h(n). Two optimal paths, both cost 8: $S \to A \to C \to G$ and $S \to B \to C \to G$.

> **Q2 (i)** Trace A* search from S to G step by step, showing the open list (with $f = g + h$ for every node considered) and the closed list after each expansion. Report the optimal path and its cost.

STEP 0 True costs h^* (needed for (ii); compute backward from G)

$h^*(G)=0$ · $h^*(C)=2$ · $h^*(B)=1+2=3$ · $h^*(A)=\min(7,\ 4+2)=6$ · $h^*(S)=\min(2+6,\ 5+3)=\mathbf{8}$

STEPS (i) A* trace, $f = g + h$ (ties broken by smaller h)

Step	Expand	New / updated nodes	OPEN after (sorted by f)	CLOSED
0	—	S: g0 h6 f6	S(6)	{ }
1	S (f=6)	A: g2+4=6 · B: g5+3=8	A(6), B(8)	{S}
2	A (f=6)	C: g=2+4=**6**, f=6+2=**8** · G: g=2+7=**9**, f=**9**	C(8,h2), B(8,h3), G(9)	{S,A}
3	C (f=8)	G: g=6+2=8 < 9, so update G to f=8 (parent C)	G(8,h0), B(8,h3)	{S,A,C}
4	G (f=8)	goal test succeeds	B(8) left	{S,A,C,G}

Optimal path $S \to A \to C \to G$, cost 2+4+2 = 8. If ties are broken FIFO (B before C), A* expands B at step 3. B finds C with g=6, which is no better, so nothing changes, and the result is the same: cost 8.

> **Q2 (ii)** Verify whether the given heuristic is admissible and whether it is consistent (monotonic) along every edge. Show your working for each edge.

STEPS (ii) Admissibility check: $h(n) \le h^*(n)$

Node	h	h^*	Check
S	6	8	**$6 \le 8$ ✓**
A	4	6	$4 \le 6$ ✓
B	3	3	$3 \le 3$ ✓ (exact)
C	2	2	$2 \le 2$ ✓ (exact)
G	0	0	$0 \le 0$ ✓

STEPS (ii) Consistency check on every edge: $h(u) \le c(u,v) + h(v)$

Edge	c	Inequality	Status
$S \to A$	2	$6 \le 2 + 4 = 6$	✓ (tight)
$S \to B$	5	$6 \le 5 + 3 = 8$	✓
$A \to C$	4	$4 \le 4 + 2 = 6$	✓
$A \to G$	7	$4 \le 7 + 0 = 7$	✓
$B \to C$	1	$3 \le 1 + 2 = 3$	✓ (tight)
$C \to G$	2	$2 \le 2 + 0 = 2$	✓ (tight)
$A \to S$	2	$4 \le 2 + 6 = 8$	✓
$B \to S$	5	$3 \le 5 + 6 = 11$	✓
$C \to A$	4	$2 \le 4 + 4 = 8$	✓
$G \to A$	7	$0 \le 7 + 4 = 11$	✓
$C \to B$	1	$2 \le 1 + 3 = 4$	✓
$G \to C$	2	$0 \le 2 + 2 = 4$	✓

The edges are written S-A, S-B, … (two-way roads), so every edge is checked in **both directions**: 12 checks, all hold. Together with $h(G)=0$, **h is consistent, and therefore also admissible.** (If the edges were one-way, the first 6 rows alone would be enough; the answer does not change.)

> **Q2 (iii)** Trace Greedy Best-First Search (ranking nodes by h(n) alone) on the same graph. Does it find the optimal path? Explain the key difference in what A* and Greedy Best-First each optimize for during node selection.

STEPS (iii) Greedy Best-First trace, f = h only

Step	Expand	OPEN after (by h)	CLOSED
0	—	S(6)	{ }
1	S	B(3), A(4)	{S}
2	B	C(2), A(4)	{S,B}
3	C	G(**0**), A(4)	{S,B,C}
4	G	goal	{S,B,C,G}

GBFS path S→B→C→G, cost 5+1+2 = 8. This happens to be optimal, but only by luck: both routes cost 8. GBFS is **not optimal in general**. For example, change B→C to cost 4. h(B)=3 is still admissible, but GBFS still goes S→B→C→G = 11, while A* finds S→A→C→G = 8.

COMPARE What each algorithm optimises

- **A*** ranks nodes by **f(n) = g(n) + h(n)**, the estimated cost of the *whole* solution through n. It balances the cost already paid against the cost still to pay. With an admissible h it is **optimal**.
- **GBFS** ranks nodes by **h(n)** alone, i.e. how close n *looks* to the goal. It ignores g(n), the cost already paid, so it is often fast but **not optimal**. It can also get trapped by misleading h values.

Memory trick: consistency = "h never drops by more than the step cost". A = "wallet + map"; GBFS = "map only".*

EXAM-READY ANSWER

Justification: On an optimal path $n=v_0 \ldots v_k=G$, applying $h(v_i) \le c(v_i,v_{i+1})+h(v_{i+1})$ repeatedly gives $h(n) \le \Sigma c + h(G) = h^*(n)$. So consistent ⇒ admissible. The converse fails: for $A \to B(1) \to G(10)$ with $h(A)=10$ and $h(B)=1$, h is admissible ($10 \le 11$, $1 \le 10$) but $10 > 1+1$ on edge $A \to B$.

(i) Expand S → OPEN A(6), B(8). Expand A → C(g6,f8), G(g9,f9). Expand C → G improves to g8, f8. Expand G. Path $S \to A \to C \to G$, cost 8.

(ii) h^* = (S8, A6, B3, C2, G0); $h \le h^*$ at every node, so h is admissible. $S \to A$ $6 \le 6$, $S \to B$ $6 \le 8$, $A \to C$ $4 \le 6$, $A \to G$ $4 \le 7$, $B \to C$ $3 \le 3$, $C \to G$ $2 \le 2$, and the reverse directions $A \to S$ $4 \le 8$, $B \to S$ $3 \le 11$, $C \to A$ $2 \le 8$, $G \to A$ $0 \le 11$, $C \to B$ $2 \le 4$, $G \to C$ $0 \le 4$ also hold, so h is consistent.

(iii) GBFS: S → B(3) → C(2) → G(0). Path $S \to B \to C \to G$, cost 8, which is optimal here only by chance. A^* minimises g+h (total estimated cost) and is optimal with admissible h. GBFS minimises h only (it ignores path cost) and is not optimal in general.

Q3 · PDDL: coffee-delivery robot

QUESTION 3 · [15 MARKS]

3. Write the PDDL domain and problem files for the following toy task, then complete the remaining parts. "A delivery robot is in the kitchen holding nothing. There is a coffee cup on the kitchen table. The goal is for the robot to be in the office holding the coffee cup."

MEANING What is being asked

PDDL splits a planning task into two files. The **domain file** holds the general rules (types, predicates, action schemas). The **problem file** holds one concrete situation (objects, initial state, goal). Grounding both gives a STRIPS task $\Pi = \langle P, A, c, I, G \rangle$: P = ground atoms, A = ground actions with pre/add/del, I = the initial atoms, G = the goal atoms. Every atom not listed in :init is false (closed-world assumption).

RULE STRIPS semantics

$$a \text{ is applicable in } s \Leftrightarrow \mathrm{pre}(a) \subseteq s \quad \cdot \quad s[\![a]\!] = (s \setminus \mathrm{del}(a)) \cup \mathrm{add}(a) \quad \cdot \quad s \text{ is a goal state} \Leftrightarrow G \subseteq s$$

Q3 (i) Define the domain file with object types, predicates, and three STRIPS actions — move(?from ?to), pick-up(?obj ?loc), and put-down(?obj ?loc) — each with explicit :precondition and :effect. Define the problem file with :objects, :init, and :goal.

ANSWER (i) Domain file

```
(define (domain delivery)
  (:requirements :strips :typing)
  (:types location item)
  (:predicates
     (robot-at ?l - location)
     (obj-at ?o - item ?l - location)
     (holding ?o - item)
     (hand-empty))
```

```
  (:action move
     :parameters (?from - location ?to - location)
     :precondition (robot-at ?from)
     :effect (and (robot-at ?to)
                  (not (robot-at ?from))))

  (:action pick-up
     :parameters (?obj - item ?loc - location)
     :precondition (and (robot-at ?loc) (obj-at ?obj ?loc) (hand-empty))
     :effect (and (holding ?obj)
                  (not (obj-at ?obj ?loc))
                  (not (hand-empty))))

  (:action put-down
     :parameters (?obj - item ?loc - location)
     :precondition (and (robot-at ?loc) (holding ?obj))
     :effect (and (obj-at ?obj ?loc)
                  (hand-empty)
                  (not (holding ?obj))))
)  ; this one bracket closes the whole domain
```

ANSWER (i) Problem file

```
(define (problem bring-coffee)
  (:domain delivery)
  (:objects kitchen office - location
            cup - item)
  (:init (robot-at kitchen)
         (obj-at cup kitchen)       ; the cup is on the kitchen table
         (hand-empty))
  (:goal (and (robot-at office)
              (holding cup)))
)
```

> **Q3 (ii)** *Give a valid plan (sequence of ground actions) and prove its correctness by showing the state produced after each action, ending in a state that satisfies the goal.*

STEPS (ii) Plan and proof of correctness

Plan π = ⟨pick-up(cup, kitchen), move(kitchen, office)⟩

Step	pre ⊆ s ?	del	add	Resulting state
s_0	—	—	—	{robot-at kitchen, obj-at cup kitchen, hand-empty}
1. pick-up(cup, kitchen)	robot-at kitchen ✓, obj-at cup kitchen ✓, hand-empty ✓	obj-at cup kitchen, hand-empty	holding cup	s_1 = {robot-at kitchen, holding cup}
2. move(kitchen, office)	robot-at kitchen ✓	robot-at kitchen	robot-at office	s_2 = {robot-at office, holding cup}

Goal check: G = {robot-at office, holding cup} ⊆ s_2 ✓. So π is a valid plan, with cost 2.

Why 2 is optimal: the goal has two atoms that are false in s_0. *holding cup* can only be added by pick-up, and *robot-at office* can only be added by move. These are two different actions, so every plan needs at least 2 actions.

Why the order matters: if you move first, the robot is in the office and the cup is in the kitchen, so pick-up is not applicable.

> **Q3 (iii)** Propose one admissible heuristic a planner could use to guide search toward the goal for this task, and briefly justify why it never overestimates the true remaining cost.

STEPS (iii) An admissible heuristic: goal count

$$h(s) = |G \setminus s| \quad \text{(the number of goal atoms not yet true in s)}$$

Why it never overestimates: look at the add lists. move adds only robot-at(?to), which is at most 1 goal atom. pick-up adds only holding(?obj), which is at most 1 goal atom. put-down adds no goal atom. So every action makes at most one missing goal atom true. With unit costs, reaching the goal needs at least $|G \setminus s|$ actions, so $h(s) \le h^*(s)$ for every state s. It is also **goal-aware** ($h = 0$ exactly when $G \subseteq s$).

Example: $h(s_0) = 2 = h^*(s_0)$, $h(s_1) = 1 = h^*(s_1)$, $h(s_2) = 0$.

Warning: goal count is **not** admissible in general. It works here only because no action adds two goal atoms at once.

Memory trick: domain = "rulebook", problem = "today's game". Apply an action as "delete first, then add": $s[\![a]\!] = (s \setminus del) \cup add$.

EXAM-READY ANSWER

(i) Domain and problem files as written above: types location/item; predicates robot-at, obj-at, holding, hand-empty; move, pick-up and put-down with the preconditions and effects shown. Init {robot-at kitchen, obj-at cup kitchen, hand-empty}; goal {robot-at office, holding cup}.

(ii) π = ⟨pick-up(cup,kitchen), move(kitchen,office)⟩. s_1 = {robot-at kitchen, holding cup}; s_2 = {robot-at office, holding cup} $\supseteq G$, so π is valid. It is optimal (cost 2), because the two goal atoms are added by two different actions.

(iii) $h(s) = |G \setminus s|$. Every action adds at most one goal atom and costs 1, so at least $|G \setminus s|$ actions are needed: $h \leq h^*$, i.e. h is admissible (and goal-aware).

Q4 · Identity vs Perfect abstraction; orthogonal abstractions

QUESTION 4 · [15 MARKS]

4. Distinguish between identity abstraction and perfect abstraction. Consider the following cleaning-robot task with three state variables: robot-location ∈ {dock, room1, room2}, room1-status ∈ {dirty, clean}, room2-status ∈ {dirty, clean}. Initial state: robot at dock, both rooms dirty. Goal: both rooms clean and robot back at dock. Actions (cost 1): move(dock,room1), move(room1,dock), move(dock,room2), move(room2,dock), move(room1,room2), move(room2,room1), clean1 (pre: robot at room1; effect: room1-status:=clean), clean2 (pre: robot at room2; effect: room2-status:=clean).

> **Q4 (first part)** Distinguish between identity abstraction and perfect abstraction.

BACKGROUND

Abstraction basics

- **Abstraction α** maps each concrete state to an abstract state. $h^{\alpha}(s)$ = cost of the cheapest abstract path from $\alpha(s)$ to an abstract goal. This value is admissible and consistent.
- A good abstraction needs two things: an **accurate** heuristic (small search) and **cheap** computation (building the heuristic must not cost too much).
- **One-state abstraction**: every state maps to the same single abstract state, so $h^{\alpha} = 0$. It is useless.
- **Identity abstraction**: $\alpha(s) = s$, so $h^{\alpha} = h^{*}$. Nothing is gained, because it is as big as the real problem.
- **Perfect abstraction**: $h^{\alpha} = h^{*}$ (for example $\alpha(s) = h^{*}(s)$). It is small, but building it requires solving the problem first.
- **Practical abstractions**: pattern databases (PDB = projection onto a subset of variables, with all abstract goal distances precomputed and stored) and merge-and-shrink.
- If abstractions are **pairwise orthogonal**, their **sum** is still safe, goal-aware, admissible and consistent.

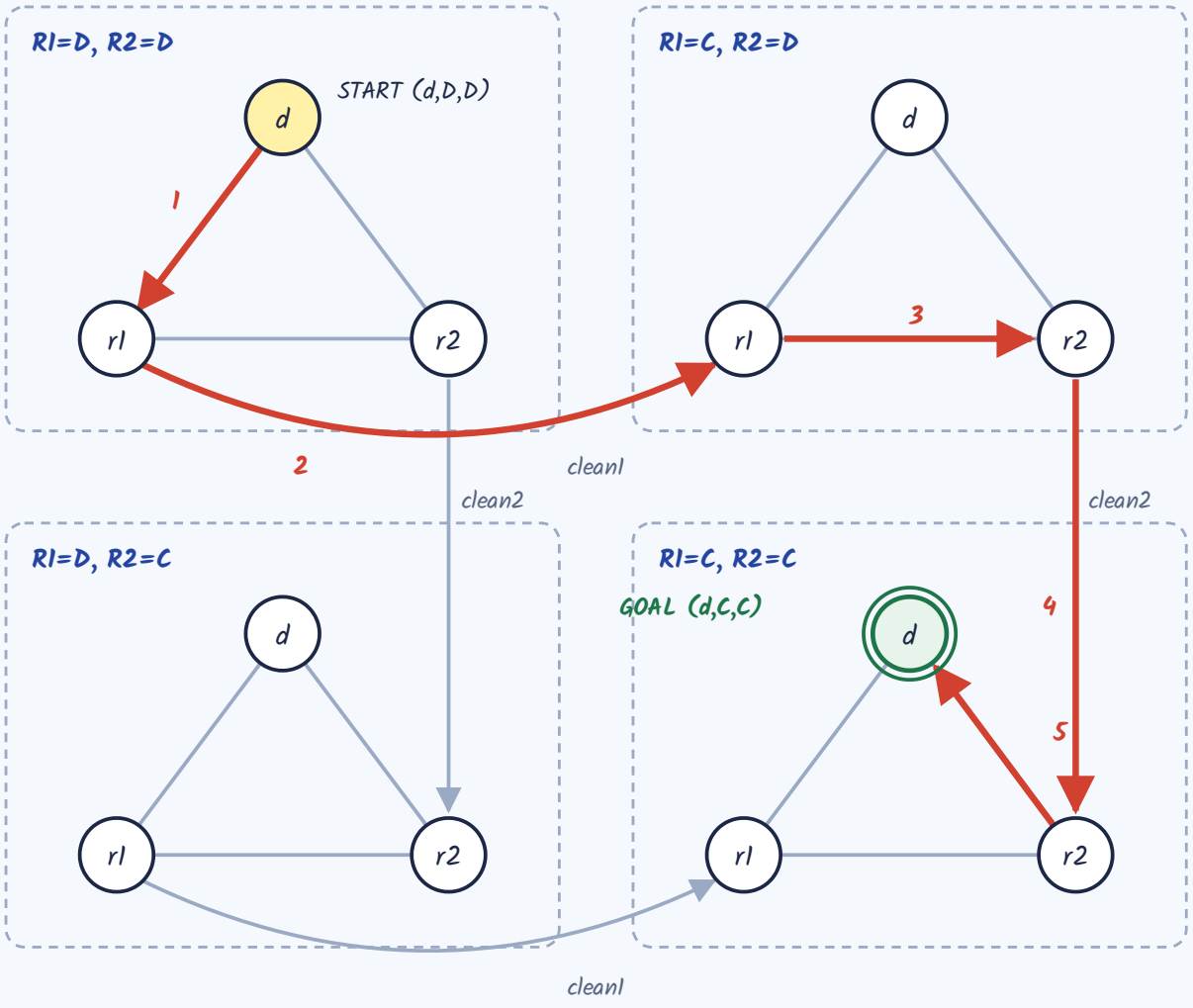
- **Spurious path**: an abstract path with no matching concrete path. It is the reason h^α underestimates.

COMPARE Identity vs Perfect abstraction

Feature	Identity abstraction	Perfect abstraction
Mapping	$\alpha(s) = s$	any α with $h^\alpha = h^*$, e.g. $\alpha(s) = h^*(s)$
Information lost	none	only what does not affect the cost to goal
Heuristic	exact, $h^\alpha = h^*$	exact, $h^\alpha = h^*$
Abstract size	= concrete size (huge)	can be very small
Problem	no reduction: computing it = solving the task	to build it you must already know h^*

> **Q4 (i)** Draw the complete concrete transition system over all reachable combinations of the three state variables, and find the actual optimal solution cost.

FIGURE (i) The complete concrete transition system: 3 × 2 × 2 = 12 states, all reachable



Each grey line inside a box = move, both directions, cost 1 (6 move transitions per box). clean1 goes right (only at r1), clean2 goes down (only at r2). Self-loops (cleaning an already clean room) are omitted. Red 1→5 = optimal plan.

Transition count: 12 states × 2 moves = 24 move transitions, plus 2 clean1 and 2 clean2 state-changing transitions (plus self-loops when the room is already clean).

Optimal plan: move(d,r1) → clean1 → move(r1,r2) → clean2 → move(r2,d). **$h^*(1) = 5$.**

Why 5 is minimal: you need 2 cleans, and you must visit r1 and r2 and return to d. That needs at least 3 moves (d→r1→r2→d, or the reverse). 2 + 3 = 5.

> **Q4 (ii)** *Find two orthogonal abstractions of the transition system (e.g., one that drops room2-status, one that drops room1-status), and calculate the estimated cheapest solution cost of each abstraction. Then admissibly combine those two abstractions by summation, and compare the combined estimate to the true optimal cost found in (i).*

STEPS (ii) The two abstractions (projections)

α_1 drops R2, so it keeps {L, R1}. *Abstract init (d,D), goal (d,C). Cheapest path: move(d,r1) → clean1 → move(r1,d).* **$h^{\alpha_1}(I) = 3$.** ✓

α_2 drops R1, so it keeps {L, R2} *(this is the literal reading of the question). Init (d,D), goal (d,C). Path: move(d,r2) → clean2 → move(r2,d).* **$h^{\alpha_2}(I) = 3$.**

RULE When can we add abstraction heuristics?

> α affects action a $\Leftrightarrow$ a causes a non-self-loop transition in α's abstract system.
>
> α_1 and α_2 are **orthogonal** $\Leftrightarrow$ no action affects both. Then $h^{\alpha_1} + h^{\alpha_2}$ is admissible and consistent.

Check the pair {L,R1} and {L,R2}: every ***move*** *action changes L, and* ***both*** *abstractions keep L. So the moves affect both, and* ***the pair is NOT orthogonal****. Summing gives 3 + 3 = 6 > h^* = 5:* ***inadmissible****. The moves are counted twice.*

Make them orthogonal: *also drop L from the second one. α_2' keeps only {R2}. Now:*

- *Active(α_1) = {all 6 move actions, clean1}*
- *Active(α_2') = {clean2}*
- *Active(α_1) ∩ Active(α_2') = ∅, so the pair* ***is orthogonal*** ✓

$h^{\alpha_2'}(I)$: D → clean2 → C, cost ***1*** *(location is ignored, so clean2 can be applied directly).*

> $h_{sum}(I) = h^{\alpha_1}(I) + h^{\alpha_2'}(I) = 3 + 1 = \mathbf{4 \le h^*(I) = 5}$ ✓ admissible (underestimates by 1)

Why the gap of 1? *α_2' does not see the move into room2 and back, and α_1 does not need move(r1,r2). An alternative that is also admissible is cost partitioning: give each move half its cost in each of {L,R1} and {L,R2}. Then h_1 = 0.5+1+0.5 = 2, h_2 = 2, and the sum is 4.*

Memory trick: "Orthogonal = no shared worker." If one action does useful work in both abstractions, you pay it twice, and the sum lies.

EXAM-READY ANSWER

Identity $\alpha(s)=s$: no information lost, $h^{\alpha}=h^*$, but the abstract space is as large as the concrete one, so there is no gain. ***Perfect*** abstraction: $h^{\alpha}=h^*$ with a possibly tiny space (e.g. $\alpha(s)=h^*(s)$), but building it requires knowing h^*, i.e. solving the task.

(i) $3\times2\times2 = 12$ states, all reachable (drawn as 4 groups by (R1,R2); moves inside a group, clean1/clean2 between groups). Optimal plan: move(d,r1), clean1, move(r1,r2), clean2, move(r2,d). $h^*(I) = 5$.

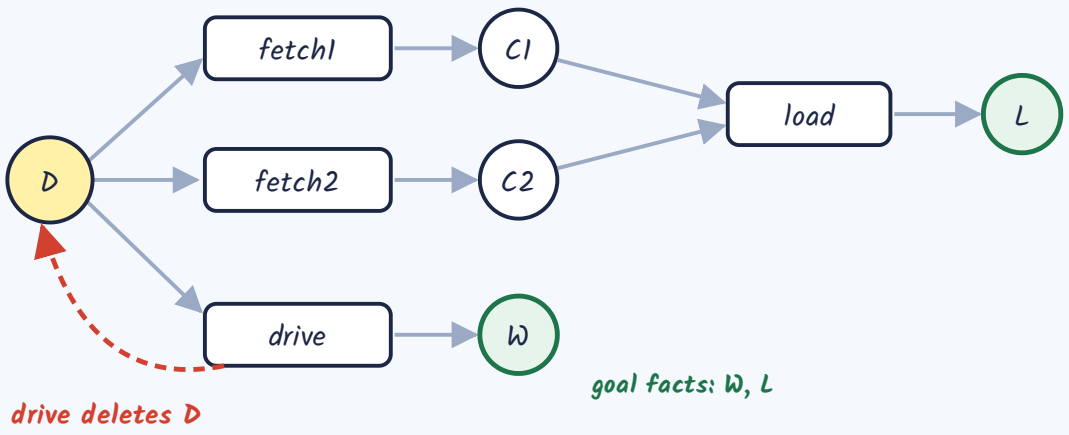
(ii) $\alpha_1 = \pi_{\{L,R1\}}$: $(d,D)\to(r1,D)\to(r1,C)\to(d,C)$, $h=3$. Keeping L in the second projection ($\pi_{\{L,R2\}}$, $h=3$) is not orthogonal (moves affect both), so $3+3=6 > 5$ would be inadmissible. Take $\alpha_2 = \pi_{\{R2\}}$: $D\to C$ by clean2, $h=1$. Active(α_1) = {moves, clean1}, Active(α_2) = {clean2}, which are disjoint, so the pair is orthogonal and $h = 3+1 = 4 \le h^* = 5$: admissible, underestimating by 1.

Q5 · Critical-path heuristics: h^{m+1} dominates h^m

QUESTION 5 · [15 MARKS]

5. *"h^{m+1} dominates h^m" — justify this statement for the critical-path heuristic family. Consider the following STRIPS task.*

```
Facts: at-dock, have-crate1, have-crate2, loaded, at-warehouse
Initial: {at-dock}            Goal: {at-warehouse, loaded}
Actions (cost 1):
  fetch-crate1: pre {at-dock}, add {have-crate1}
  fetch-crate2: pre {at-dock}, add {have-crate2}
  load:  pre {have-crate1, have-crate2}, add {loaded}
  drive: pre {at-dock}, del {at-dock}, add {at-warehouse}
```



The conflict: both fetches need D, and drive destroys D. So the real plan must be fetch1, fetch2, (load), drive. $h^* = 4$.

RULE The h^m equations

$h^m(B) = 0$ if $B \subseteq s$

$h^m(B) = \min_{a:\ \mathrm{regr}(B,a)\ \mathrm{defined}} [\ c(a) + h^m(\mathrm{regr}(B,a))\ ]$ if $|B| \le m$

$h^m(B) = \max_{B' \subseteq B,\ |B'| = m} h^m(B')$ if $|B| > m$

$\mathrm{regr}(B,a) = (B \setminus \mathrm{add}(a)) \cup \mathrm{pre}(a)$, defined only if $\mathrm{add}(a) \cap B \neq \emptyset$ and $\mathrm{del}(a) \cap B = \emptyset$

$h^m(s) = h^m(G)$. For $m = 1$ this is exactly h^{max}.

> **Q5 (i)** Compute $h^1(s0)$ (the h_max critical-path heuristic) for the goal, showing the fixpoint computation fact-by-fact.

STEPS (i) $h^1(s_0)$: fixpoint iteration

Fact	it 0	it 1	it 2	it 3	How the value is obtained
D	0	0	0	0	true in s_0
C1	∞	1	1	1	fetch-crate1: 1 + h^1(D) = 1
C2	∞	1	1	1	fetch-crate2: 1 + h^1(D) = 1
W	∞	1	1	1	drive: 1 + h^1(D) = 1
L	∞	∞	2	2	load: 1 + max(h^1(C1), h^1(C2)) = 1 + 1 = 2

Iteration 3 = iteration 2, so the fixpoint is reached. **$h^1(s_0)$ = max(h^1(W), h^1(L)) = max(1, 2) = 2.**

True cost: fetch1, fetch2, load, drive. The drive must come last, because it deletes D, which both fetches need. **$h^*(s_0) = 4$.**

Why h^1 is so low: at load, max(C1, C2) = 1 pretends that one fetch gives both crates. At the goal, max(W, L) pretends that the drive costs nothing extra on top of the loading chain.

> **Q5 (ii)** Compute $h^2(s0)$ for the pair {loaded, at-warehouse} using the standard h^m recursion, and compare it to $h^1(s0)$. Does h^2 recover the true optimal plan cost here? Show your working.

STEPS (ii) $h^2(s_0)$: every set of size ≤ 2 that is needed

Set B	Possible last actions → regression	h^2
{D}	true in s_0	0
{C1}, {C2}, {W}	fetch1 / fetch2 / drive → {D}	1
{C1,D}	fetch1 → {D} (fetch does not delete D)	1
{C2,D}	fetch2 → {D}	1
{W,D}	drive adds W but **deletes D**, so it cannot be last; no action adds D	∞
{C1,C2}	fetch1 → {C2,D} = 1; fetch2 → {C1,D} = 1	2
{W,C1}	drive → {C1,D} = 1 → 2; fetch1 → {W,D} = ∞	2
{W,C2}	drive → {C2,D} = 1 → 2	2
{L,D}	load → {C1,C2,D}: size 3 → max({C1,C2}=2, {C1,D}=1, {C2,D}=1) = 2 → 1+2	3
{L}	load → {C1,C2} = 2 → 1+2	3
{L,C1}, {L,C2}	load → {C1,C2} = 2 → 3 (fetch last → {L,D} = 3 → 4)	3

Goal {L, W}:

Last action	Regression	Value	Total
load	{W, C1, C2} (size 3)	max({W,C1}=2, {W,C2}=2, {C1,C2}=2) = 2	1+2 = 3
drive	{L, D}	3	1+3 = 4

$$\mathbf{h^2(s_0) = \min(3, 4) = 3} \quad h^1 = 2 \le h^2 = 3 \le h^* = 4$$

Does h^2 recover h^*? No. h^2 sees that the two crates need **two** fetches ({C1,C2} = 2). But {W,C1,C2} is a triple. h^2 splits it into pairs and never sees that **both** fetches must happen before the drive.

STEPS $h^3(s_0) = 4$: triples recover the optimum

- $\{C1,C2,D\}$: fetch1 → $\{C2,D\} = 1$ → **2**
- $\{W,C1,C2\}$: drive → $\{C1,C2,D\} = 2$ → **3**. fetch1 → $\{W,C2,D\} = \infty$ (W and D can never hold together).
- $\{L,D\}$: load → $\{C1,C2,D\} = 2$ → **3**
- Goal $\{L,W\}$: load → $\{W,C1,C2\} = 3$ → **4**; drive → $\{L,D\} = 3$ → 4. **$h^3(s_0) = 4 = h^*$** ✓

> **QS (iii)** Using this example, explain in general terms why $h^{m+1}(s) \geq h^m(s)$ always holds — i.e., why reasoning about larger fact-sets jointly can never produce a lower estimate than reasoning about smaller subsets.

PROOF (iii) Why $h^{m+1}(s) \geq h^m(s)$ always holds

Lemma (monotonicity): for every m, if $B' \subseteq B$ then $h^m(B') \leq h^m(B)$. Reaching all of B also reaches all of B', so a bigger set can never be cheaper. (Formally, by induction over the fixpoint iterations: any regression that is legal for B gives a superset of a legal regression for B'.)

Proof of dominance (induction over the fixpoint iterations, starting from the same values 0 / ∞):

1. **Sets of size ≤ m:** h^m and h^{m+1} use the **same** regression rule, $\min_a[c(a) + h(\text{regr}(B,a))]$.
2. **The only difference** is how a regressed set R with $|R| > m$ is valued. h^m uses $\max_{R' \subseteq R,\ |R'|=m} h^m(R')$. h^{m+1} uses R's own value if $|R| = m+1$, or the max over its $(m+1)$-subsets if $|R| > m+1$. Either way, every m-subset R' lies inside what h^{m+1} evaluates, so by the lemma $h^{m+1}(R) \geq \max_{R'} h^{m+1}(R')$, and by the induction hypothesis this is $\geq \max_{R'} h^m(R')$.
3. So in every iteration, each value h^{m+1} computes is ≥ the matching h^m value. This carries over to the fixpoint: **$h^{m+1}(s) \geq h^m(s)$ for every s.**

In words: h^m assumes that the most expensive m-subset alone decides the cost. h^{m+1} is allowed to see interactions between m+1 facts, which can only add cost, never remove it. Every h^m is admissible, so:

$h^1 = h^{max} \le h^2 \le \ldots \le h^m \le h^*$, and $h^m = h^*$ once m ≥ the number of facts (here: $2 \le 3 \le 4 = 4$)

Memory trick: "Bigger glasses never shrink the bill." h^m looks at the world through windows of size m. A bigger window shows more conflicts, so the cost estimate can only stay the same or go up.

EXAM-READY ANSWER

(i) h^1: D=0; C1=C2=W=1 (one action from D); L = 1+max(1,1) = 2. The fixpoint is reached at iteration 2. $h^1(s_0)$ = max(W,L) = 2. (h^* = 4: fetch1, fetch2, load, drive.)

(ii) h^2: {C1,C2}=2, {W,C1}={W,C2}=2, {W,D}=∞ (drive deletes D), {L,D}=3. Goal {L,W}: load → {W,C1,C2} = max of pairs = 2 → 3; drive → {L,D}=3 → 4. $h^2(s_0)$=3. So h^1=2 ≤ h^2=3 < h^*=4: h^2 does not recover h^*. It needs triples: h^3 = 4, since {W,C1,C2} = 3.

(iii) h^m and h^{m+1} use the same regression for sets of size ≤ m. The only difference is that h^m values any set larger than m by the max over its m-subsets, while h^{m+1} looks at (m+1)-subsets (which contain those m-subsets). By monotonicity (a superset costs at least as much as any subset), h^{m+1}'s value is ≥ that max. By induction over the fixpoint iterations, $h^{m+1}(s) \ge h^m(s)$ for all s. Hence $h^1 \le h^2 \le \ldots \le h^*$.

Q6 · Goal-aware vs safe; h^{max}/h^{add}; only-adds relaxation

QUESTION 6 · [15 MARKS]

6. What is the relationship between goal-aware and safe heuristics? Consider the following logistics problem.

```
Initial state I: t(B), p(A).        Goal G: t(C), p(D).
Locations, left to right: A – B – C – D
Actions A: dr(X,Y) drive truck between adjacent locations, cost 1;
           lo(X) load package (truck and package both at X), cost 1;
           ul(X) unload package (truck at X, package in truck), cost 1.
```

> **Q6 (first part)** What is the relationship between goal-aware and safe heuristics?

RULE **Heuristic properties**

- **Safe:** $h(s) = \infty \Rightarrow h^*(s) = \infty$. It never calls a solvable state a dead end.
- **Goal-aware:** $h(s) = 0$ for every goal state s.
- **Admissible:** $h(s) \le h^*(s)$. **Consistent:** $h(s) \le c(a) + h(s[\![a]\!])$.

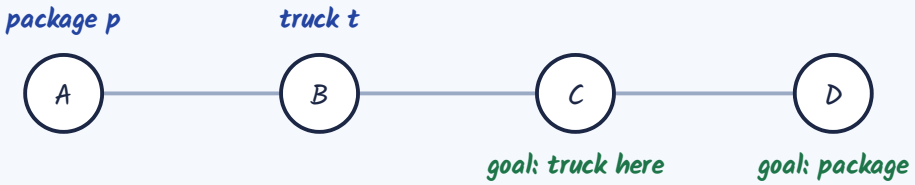
ANSWER **Relationship between goal-aware and safe**

They are independent: neither implies the other.

- $h(s) = 1$ for all s: **safe** (never ∞) but **not goal-aware** ($h = 1$ on goal states).
- $h(s) = 0$ on goal states and ∞ elsewhere: **goal-aware** but **not safe** (solvable non-goal states get ∞).

Links to other properties:

- Admissible $\Rightarrow$ safe and goal-aware. For a goal s, $h(s) \le h^*(s) = 0$. If $h(s) = \infty$, then $h^*(s) \ge \infty$.
- Goal-aware + consistent $\Rightarrow$ admissible.
- Goal-aware + safe does **not** imply admissible. Example: $h = 0$ on goals and 100 elsewhere.



> **Q6 (i)** Compute the heuristic value of the initial state using delete relaxation (i.e., h_add or h_max over the relaxed planning graph — state which one you use and why). Show the relaxed planning graph layer by layer.

STEPS (i) Relaxed planning graph, layer by layer

F = fact layer, A = action layer. **Bold** = new in this layer; grey = carried over (in the relaxation, facts are never deleted). p(T) means "package in truck".

F0
t(B)
p(A)

A0
dr(B,A)
dr(B,C)

F1
t(B), p(A)
t(A)
t(C)

A1
A0 +
lo(A)
dr(A,B)
dr(C,B)
dr(C,D)

F2
F1 +
p(T)
t(D)

A2
A1 +
ul(A), ul(B)
ul(C), ul(D)
dr(D,C)

F3
F2 +
p(B), p(C)
p(D)
G ⊆ F3 → stop

STEPS (i) Fact costs: h^{max} and h^{add}

$h^{max}(p) = \min_{a \text{ achieves } p} [\, c(a) + \max_{q \in pre(a)} h(q) \,]$ · $h^{add}(p) = \min_{a} [\, c(a) + \Sigma_{q \in pre(a)} h(q) \,]$

Fact	Best achiever	pre	h^{max}	h^{add}
t(B), p(A)	—	—	0	0
t(A)	dr(B,A)	t(B)	1	1
t(C)	dr(B,C)	t(B)	1	1
t(D)	dr(C,D)	t(C)	2	2
p(T)	lo(A)	t(A), p(A)	1+max(1,0) = 2	1+(1+0) = 2
p(D)	ul(D)	t(D), p(T)	1+max(2,2) = 3	1+(2+2) = 5
Goal {t(C), p(D)}	combine the goal facts		**max(1,3) = 3**	**1+5 = 6**

Compare with the exact values:

- Optimal relaxed plan: dr(B,A), lo(A), dr(B,C), dr(C,D), ul(D), so **$h^+(I) = 5$**.
- Real optimal plan: dr(B,A), lo(A), dr(A,B), dr(B,C), dr(C,D), ul(D), dr(D,C), so **$h^*(I) = 7$**.

$$h^{max} = 3 \leq h^+ = 5 \leq h^* = 7 \quad \text{while } h^{add} = 6 > h^+ = 5$$

Which one to use and why: use **h^{max}**. It is admissible ($h^{max} \leq h^+ \leq h^*$), so A^* stays optimal. h^{add} is more informative but **not admissible**, because it adds costs of subgoals that share actions. Here **dr(B,C) is counted twice**: once inside t(C) = 1 and again inside t(D) = 2 (needed for p(D)). So $h^{add} = 6 > h^+ = 5$. It happens to stay below $h^* = 7$ here, but in general it can overestimate h^*. Use h^{add} with greedy search, not with optimal A^*.

> **Q6 (ii)** Is an Only-Adds relaxation (which drops only the delete lists but keeps all preconditions exactly as in the real task) more informative than the standard delete relaxation used above? Justify your answer by identifying what Only-Adds relaxation would still correctly rule out that delete relaxation cannot.

STEPS (ii) Only-adds vs delete relaxation

Answer: No. The "only-adds" relaxation is not more informative. The question's own premise is a trap. As defined in the question, it "drops only the delete lists and keeps all preconditions". That is **exactly the standard delete relaxation**, so it gives the same value $h^+(I) = 5$ and rules out exactly the same things. There is nothing extra it could rule out.

The **usual only-adds relaxation** (from the lecture) drops the **preconditions as well as** the deletes: only the add lists matter. It is **less** informative:

- **$h^{OA}(I) = 2$** via $\langle dr(B,C), ul(D)\rangle$: one action adds $t(C)$ and one adds $p(D)$, and no single action adds both.
- It allows unloading at D even though the truck is not at D and the package is not in the truck.
- Delete relaxation **still enforces preconditions**, so it correctly rules out such impossible plans. Only-adds cannot.
- Therefore **$h^{OA} \le h^{+} \le h^{*}$** ($2 \le 5 \le 7$). Delete relaxation **dominates** only-adds, not the other way round.

Memory trick: relaxation ladder, from strongest to weakest: h^* (7) $\ge$ h^+ (5, deletes dropped) $\ge$ h^{OA} (2, deletes and preconditions dropped). The more you drop, the lower the value. "Max is modest, Add adds too much."

EXAM-READY ANSWER

Goal-aware vs safe: independent. $h \equiv 1$ is safe but not goal-aware. $h = 0$ on goals and ∞ elsewhere is goal-aware but not safe. Admissible $\Rightarrow$ both. Goal-aware + consistent $\Rightarrow$ admissible.

(i) RPG: F0 $\{t(B),p(A)\}$ $\rightarrow$ A0 $dr(B,A)$, $dr(B,C)$ $\rightarrow$ F1 $+t(A),t(C)$ $\rightarrow$ A1 $lo(A)$, $dr(C,D)$, … $\rightarrow$ F2 $+p(T),t(D)$ $\rightarrow$ A2 $ul(D)$, … $\rightarrow$ F3 $+p(D)$, and $G \subseteq$ F3. Costs: $t(A)=t(C)=1$, $t(D)=2$, $p(T)=2$, $p(D)=3$ (max) / 5 (add). **$h^{max}(I) = \max(1,3) = 3$**; $h^{add}(I) = 1+5 = 6$. I use h^{max} because it is admissible: $3 \le h^+ = 5 \le h^* = 7$. h^{add} double-counts $dr(B,C)$ ($6 > h^+ = 5$) and is inadmissible in general.

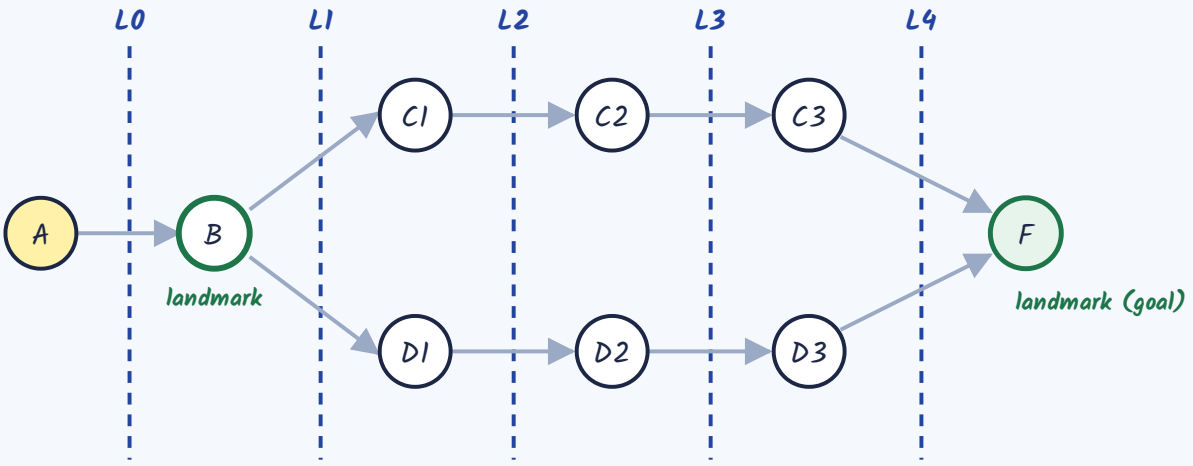
(ii) No. As described (deletes dropped, preconditions kept), it is identical to delete relaxation, so it is equally informative and rules out nothing extra. The usual only-adds relaxation also drops preconditions: $h^{OA}(I) = 2$ via $\langle dr(B,C), ul(D)\rangle$, which allows unloading at D without the package. Delete relaxation rules this out, so $h^{OA} \le h^+$: delete relaxation dominates.

Q7 · Fact landmarks, disjunctive action landmarks, LM-cut

QUESTION 7 · [15 MARKS]

7. What is a fact landmark? What are the fact landmarks and necessary subgoals for the initial state of the following path-finding problem with actions move(X, Y): pre X, eff Y; init A, goal F?

```
          C1 — C2 — C3
         /            \
A — B                  F
         \            /
          D1 — D2 — D3
```



Green = fact landmarks B, F. Blue dashed = the 5 disjunctive action landmarks (cuts). Each cut crosses every A → F route exactly once.

> **Q7 (first part)** What is a fact landmark? What are the fact landmarks and necessary subgoals for the initial state of the following path-finding problem with actions move(X, Y): pre X, eff Y; init A, goal F?

RULE Definitions (FDR task $\Pi = \langle V, A, c, I, G \rangle$, state s)

Fact landmark: a fact p is a landmark for s if for **every** plan $\langle a_1,\ldots,a_n \rangle$ for s there is a prefix length t with $p \in s[\![\langle a_1,\ldots,a_t \rangle]\!]$. In words, p becomes true at some point in every plan. It is **non-trivial** if $p \notin s$.

Disjunctive action landmark: a set $L \subseteq A$ such that every plan for s contains at least one action from L.

Necessary subgoals (backchaining): (1) every goal fact $p \notin s$ is a landmark; (2) if p is a landmark and **all** achievers of p share a precondition $q \notin s$, then q is a landmark too: $q \in \bigcap_{a:\, p \in \mathrm{eff}(a)} \mathrm{pre}(a)$.

STEPS Fact landmarks and necessary subgoals for I = {A}

Note: whether moves go only left-to-right or both ways along the drawn edges, every result below (landmarks, LM(p) values, LM-cut rounds, h = 5) is exactly the same.

- **Fact landmarks = {B, F}** (non-trivial). A is a trivial landmark, since it is already true in I. Every route from A must pass through B, and every plan ends with F true.
- **Necessary subgoals:** start with F (it is the goal). Achievers of F: move(C3,F) with pre {C3}, and move(D3,F) with pre {D3}. {C3} ∩ {D3} = ∅, so backchaining stops. **Necessary subgoals = {F}.**
- Note: B is a landmark, but backchaining **cannot** find it, because the chain breaks at F. You need delete-relaxation methods, shown below.

> **Q7 (i)** Show why naive fact-landmark extraction fails here — i.e., why neither the C-branch nor the D-branch facts qualify as a fact landmark individually, even though the goal cannot be reached without using one branch or the other.

STEPS (i) Why naive fact-landmark extraction fails

There are two alternative branches from B to F:

- C-plan: $A \to B \to C1 \to C2 \to C3 \to F$
- D-plan: $A \to B \to D1 \to D2 \to D3 \to F$

The C-plan never makes D1, D2 or D3 true, so no D-fact is a landmark. The D-plan never makes C1, C2 or C3 true, so no C-fact is a landmark. Only {B, F} survive, and counting them gives $h(I) = 2$ while $h^*(I) = 5$. Knowledge like "one of C1 or D1 must be reached" is **disjunctive**, and single facts cannot express it.

> **Q7 (ii)** Explain how the problem can be rescued using fact-induced disjunctive action landmarks. Extract the disjunctive action landmarks for this problem using (a) delete relaxation and (b) LM-cut, and show the resulting landmark sets.

STEPS (ii-a) Landmarks via delete relaxation

Idea: work in the relaxed task Π^+, where deletes are ignored. A fact p is a landmark if Π^+ becomes unsolvable after removing all achievers of p. A set L of actions is a disjunctive action landmark if Π^+ becomes unsolvable after removing L.

Fixed-point equations (LM(p) = the landmarks that must hold before p):

$$\mathrm{LM}(p) = \{p\} \text{ for } p \in I; \quad \mathrm{LM}(p) = \{p\} \cup \bigcap_{a:\, p \in \mathrm{add}(a)} \bigcup_{q \in \mathrm{pre}(a)} \mathrm{LM}(q)$$

Fact	Achievers	LM(fact)
A	(initial)	{A}
B	move(A,B)	{B} ∪ LM(A) = {A,B}
C1 / C2 / C3	move(B,C1) / move(C1,C2) / move(C2,C3)	{A,B,C1} / {A,B,C1,C2} / {A,B,C1,C2,C3}
D1 / D2 / D3	move(B,D1) / move(D1,D2) / move(D2,D3)	{A,B,D1} / … / {A,B,D1,D2,D3}
F	move(C3,F), move(D3,F)	{F} ∪ (LM(C3) ∩ LM(D3)) = **{A, B, F}**

So the fact landmarks are {A, B, F}; non-trivial: {B, F}. Now turn them into action landmarks, and add every cut whose removal makes F relaxed-unreachable:

- Achievers of landmark B: **L0 = {move(A,B)}**
- Removing {move(B,C1), move(B,D1)} makes F unreachable: **L1 = {move(B,C1), move(B,D1)}**
- **L2 = {move(C1,C2), move(D1,D2)}**
- **L3 = {move(C2,C3), move(D2,D3)}**
- Achievers of landmark F: **L4 = {move(C3,F), move(D3,F)}**

STEPS (ii-b) Landmarks via LM-cut (unit costs)

LM-cut loop: (1) compute h^{max}; stop if h^{max}(goal) = 0. (2) Each action picks its precondition with the highest h^{max} (here each action has only one precondition). (3) The **goal zone** = facts that reach the goal using 0-cost actions. (4) The **cut** = actions

that go from outside the goal zone into it; this is a landmark L. (S) Add m = min cost in L to h, subtract m from every action in L, and repeat.

Round	$h^{max}(F)$	Goal zone	Cut (landmark)	cost
1	5	{F}	L4 = {move(C3,F), move(D3,F)}	1
2	4	{F, C3, D3}	L3 = {move(C2,C3), move(D2,D3)}	1
3	3	{F, C3, D3, C2, D2}	L2 = {move(C1,C2), move(D1,D2)}	1
4	2	… ∪ {C1, D1}	L1 = {move(B,C1), move(B,D1)}	1
5	1	… ∪ {B}	L0 = {move(A,B)}	1
6	0	stop → $h^{LM\text{-}cut}(I) = 1+1+1+1+1 = \mathbf{5}$		

h^{max} in round 1: A=0, B=1, C1=D1=2, C2=D2=3, C3=D3=4, F=5. LM-cut finds the **same 5 landmarks**, but from the goal backward (L4 first). Here they are **pairwise disjoint**: every action in each cut costs exactly m = 1, so it drops to cost 0 and can never be cut again. (In general, LM-cut landmarks can overlap, when an action costs more than m. They are still admissible together, because subtracting m after each cut is a cost partitioning.)

> **Q7 (iii)** Explain the steps needed to admissibly combine the disjunctive action landmarks you found into a single heuristic estimate for the initial state (i.e., why simple counting can overcount, and what a cost-partitioning or uniqueness argument buys you here).

STEPS (iii) Combining landmarks admissibly

Elementary landmark heuristic: $h_L(s) = \min_{a \in L} c(a)$. This is admissible, because every plan pays for at least one action of L.

Why simple counting can overcount: one action can appear in several landmarks. Summing then pays for that action more than once. **Example:** L5 = {move(B,C1), move(D3,F)} is also a valid landmark, because every C-plan uses move(B,C1) and every D-plan uses move(D3,F). Summing L0…L5 gives 6 > h^* = 5, which is **not admissible**. The C-plan pays move(B,C1) once, but it was counted in both L1 and L5.

Fixes:

- **Disjoint (orthogonal) landmarks / canonical heuristic:** only sum landmarks that share no action, i.e. $L_i \cap L_j = \emptyset$. Take the max over such disjoint sets.

- **Cost partitioning:** split each action's cost among the landmarks that contain it, so that $\Sigma_i\, c_i(a) \le c(a)$. Then $h = \Sigma_i \min_{a \in L_i} c_i(a)$ is admissible (uniform or optimal (LP) cost partitioning). LM-cut does this automatically by reducing the costs after each cut.

Here: L0–L4 are pairwise disjoint (uniqueness: each action is in exactly one landmark), so each landmark can use the full cost:

$h(I) = \Sigma_{i=0..4} \min_{a \in L_i} c(a) = 1+1+1+1+1 = \mathbf{5 = h^*(I)}$ (perfect; also equals $h^+(I) = 5$)

TRICK F → N → D → C (mnemonic)

- **F** = Fact landmark: a fact that every plan must pass through (here B, F).
- **N** = Necessary subgoal: go backward from the goal and keep the precondition that all achievers share (here only F).
- **D** = Disjunctive action landmark: not one fixed action, but a set with "at least one of" (OR).
- **C** = Cut / LM-cut: the mandatory cuts that every route must cross.

Picture it: every cut is a fence across the road. Every route to F must cross each fence once. Fences that do not overlap can be added up.

EXAM-READY ANSWER

A **fact landmark** for s is a fact p that becomes true at some point in every plan from s (non-trivial if $p \notin s$). For $I=\{A\}$: fact landmarks $\{B, F\}$ (A is trivial). Necessary subgoals: $\{F\}$ only, because the achievers of F have preconditions $\{C3\}$ and $\{D3\}$, which share nothing, so backchaining stops.

(i) The C-route never visits D1–D3 and the D-route never visits C1–C3, so no single branch fact is a landmark. Only $\{B,F\}$ remain: $h=2 \ll h^*=5$. The necessary knowledge is disjunctive ("C1 or D1").

(ii) (a) Delete relaxation: $LM(F) = \{F\} \cup (LM(C3) \cap LM(D3)) = \{A,B,F\}$. Achiever sets and relaxed cuts give $L0=\{move(A,B)\}$, $L1=\{move(B,C1),move(B,D1)\}$, $L2=\{move(C1,C2),move(D1,D2)\}$, $L3=\{move(C2,C3),move(D2,D3)\}$, $L4=\{move(C3,F),move(D3,F)\}$. (b) LM-cut: $h^{max}(F)=5$ → cut L4, then L3, L2, L1, L0, each costing 1, then $h^{max}=0$. $h^{LM\text{-}cut}(I)=5$.

(iii) $h_L = \min_{a \in L} c(a)$. Summing overlapping landmarks double-counts shared actions (e.g. adding $L5=\{move(B,C1),move(D3,F)\}$ gives $6 > 5$). An admissible combination needs pairwise disjoint landmarks, or a cost partitioning with $\Sigma_i c_i(a) \leq c(a)$. L0–L4 are disjoint, so $h = 1+1+1+1+1 = 5 = h^*(I)$.

Last-night sheet

- **Admissible:** $h \le h^*$. **Consistent:** $h(u) \le c(u,v)+h(v)$ and $h(goal)=0$. Consistent ⇒ admissible (telescoping sum); the reverse does not hold.
- **A*** $f = g+h$; optimal with admissible h (graph search without reopening needs consistent h). **GBFS** $f = h$; fast, not optimal.
- **BFS**: complete (finite b), optimal only for unit costs. **DFS**: complete only on finite graphs with duplicate check; not optimal.
- **Identity** $\alpha(s)=s$: exact but no gain. **Perfect**: exact, but you need h^* to build it. **One-state**: $h=0$.
- **Orthogonal** abstractions: no action affects both, so the sum is admissible. If both keep a variable that some action changes, they are NOT orthogonal.
- **Fact landmark**: true at some point in every plan. **Disjunctive action landmark**: every plan uses at least one action of L. h_L = min cost in L.
- **Overlap ⇒ overcount.** Use disjoint landmarks or a cost partitioning. LM-cut = h^{max} → goal zone → cut → subtract cost → repeat.
- **PDDL:** domain = types + predicates + actions (:parameters, :precondition, :effect); problem = :objects, :init, :goal. Closed world. $s[\![a]\!] = (s \setminus del) \cup add$.
- **Goal count** $|G \setminus s|$ is admissible only when each action adds at most one goal atom and costs are ≥ 1.
- **h^m**: exact regression for sets of size ≤ m, max over m-subsets for larger sets. $h^1 = h^{max}$. $h^m \le h^{m+1} \le h^*$ (use ≥, never >).
- **Regression:** $regr(B,a) = (B \setminus add) \cup pre$; illegal if a deletes something in B, or adds nothing in B.
- **$h^{max} \le h^+ \le h^*$** (admissible). $h^{add} \ge h^{max}$, not admissible (it double-counts shared actions).
- **Safe**: ∞ only on real dead ends. **Goal-aware**: 0 on goals. They are independent; admissible ⇒ both.
- **Only-adds** (no preconditions, no deletes) ≤ delete relaxation ≤ h^*.